Task Breakdown: Quantitative Event Engine (Ledger Productionization)

Overview

Total Task Groups: 10 Estimated Complexity: XL (as per roadmap)

This task breakdown implements the Quantitative Event Engine - a stateless Smart Contract Adapter library in Rust that processes lifecycle events for structured products using a two-call pattern that separates product-level economics from position-level accounting.

Key Implementation Sequence: 1. Ledger Abstraction Layer MUST come before in-memory implementation 2. Thin slice focuses on continuous barrier products (Knock-Out Warrant, Mini Certificate) 3. Two-call pattern: Call 1 returns ProductAction, Call 2 returns ApplyResult (Vec + Vec) 4. Event Framework, Quant Lib, and Market are stubs only

---

Task List

Foundation Layer

Task Group 1: Shared Domain Model

Dependencies: None **Complexity:** L

- ☐ 1.0 Complete shared product/asset domain model
  - ☐ 1.1 Write 4-6 focused tests for domain types
    - Test Product enum discrimination and serialization
    - Test Asset enum discrimination and serialization
    - Test ProductId/AssetId key generation and equality
    - Test type conversions between Product and Asset (products can be traded as assets)
  - ☐ 1.2 Define Product enum with 14 variants
    - BarrierReverseConvertible, BarrierReverseConvertiblePro
    - BonusCertificate, CappedBonusCertificate, CappedBonusProCertificate
    - CappedWarrant, DiscountCertificate, KnockOutWarrant
    - ReverseCappedBonusCertificate, ReverseConvertible
    - MiniCertificate, OpenEndTurbo, FactorCertificate
    - VanillaOption
  - ☐ 1.3 Define Asset enum with 10 variants
    - Equity, Bond, Future, BarrierOption
    - InterestRateFuture, InterestRateOption
    - Cash, FxOption, FxSwap, StockBorrowLoan
  - ☐ 1.4 Define shared Instrument enum (Product | Asset)
    - Enable unified handling across Position Ledger
    - Support products-as-assets scenario
  - ☐ 1.5 Implement serde + prost serialization for all types

    - JSON serialization via serde
    - Protobuf serialization via prost (standard with quant library)
- ☐ 1.6 Ensure domain model tests pass
    - Run ONLY the 4-6 tests written in 1.1

Acceptance Criteria: - All 14 product types defined with proper discrimination - All 10 asset types defined with proper discrimination - Shared Instrument enum enables unified Position handling - JSON and Protobuf serialization working for all types

---

Task Group 2: Trigger Type System

Dependencies: Task Group 1 **Complexity:** L

- ☐ 2.0 Complete trigger type system
    - ☐ 2.1 Write 4-6 focused tests for trigger types
        - Test TriggerType enum serialization roundtrip
        - Test TriggerInfo construction for each category (time-based, price-based, exercise, corporate action)
        - Test fixing subtypes (initial, final, periodic, reset)
        - Test BarrierDirection (Up, Down) behavior
    - ☐ 2.2 Define TriggerType enum with 8 variants
        - Time-based: Expiry, CouponPayment
        - Price observation: Fixing (with FixingType subtype)
        - Discrete barrier: Barrier
        - Continuous barrier: ContinuousBarrier
        - Exercise: AmericanExercise
        - Corporate actions: Dividend, StockSplit
    - ☐ 2.3 Define FixingType enum
        - Initial, Final, Periodic, Reset variants
    - ☐ 2.4 Define TriggerInfo struct
        - trigger_type: TriggerType
        - trigger_time: DateTime
        - observed_price: Option (for fixings - passed by Event Framework)
        - is_provisional: bool (for provisional calculations)
        - barrier_level: Option
        - barrier_direction: Option
        - corporate_action_details: Option
    - ☐ 2.5 Define trigger-product mapping as configuration
        - Which triggers apply to which products
        - Reference product-to-trigger matrix from requirements
    - ☐ 2.6 Ensure trigger type tests pass
        - Run ONLY the 4-6 tests written in 2.1

Acceptance Criteria: - All 8 trigger types properly defined - Fixing subtypes support full lifecycle (initial through final) - TriggerInfo captures all required data for callback payloads - Product-trigger compatibility matrix implemented

---

Task Group 3: Ledger Abstraction Layer

Dependencies: Task Group 1 **Complexity:** M **Critical Note:** This MUST be implemented BEFORE the in-memory ledger

- [ ] 3.0 Complete ledger abstraction layer
  - [ ] 3.1 Write 4-6 focused tests for ledger traits
    - Test trait object creation and basic method signatures
    - Test Position struct construction with tags
    - Test Move struct construction with inherited tags
    - Test LedgerError variants
  - [ ] 3.2 Define Position struct
    - id: PositionId
    - wallet_id: WalletId
    - instrument: Instrument (from Group 1)
    - quantity: Decimal
    - tags: HashMap<TagKey, TagValue> (for portfolio system)
  - [ ] 3.3 Define Move struct
    - id: MoveId
    - position_id: PositionId
    - move_type: MoveType
    - debit_account: Account
    - credit_account: Account
    - amount: Decimal
    - effective_date: Date
    - source_action: ProductActionRef (audit trail)
    - rule_applied: MoveRuleRef (audit trail)
    - context_snapshot: HashMap<String, String>
    - inherited_tags: HashMap<TagKey, TagValue>
  - [ ] 3.4 Define LedgerReader trait
    - get_position(&self, id: &PositionId) -> Result
    - get_positions_by_wallet(&self, wallet_id: &WalletId) -> Result<Vec>
    - get_balance(&self, position_id: &PositionId) -> Result
    - positions_by_portfolio(&self, filter: &PortfolioFilter) -> Result<Vec>
    - moves_by_portfolio(&self, filter: &PortfolioFilter) -> Result<Vec>
  - [ ] 3.5 Define LedgerWriter trait
    - create_position(&mut self, position: Position) -> Result
    - record_moves(&mut self, moves: Vec) -> Result<()>
  - [ ] 3.6 Define Ledger trait (combines LedgerReader + LedgerWriter)
    - Trait bounds: LedgerReader + LedgerWriter
    - Enable future persistent implementations
  - [ ] 3.7 Ensure ledger abstraction tests pass
    - Run ONLY the 4-6 tests written in 3.1

Acceptance Criteria: - Ledger trait is storage-agnostic - Position and Move structs support full tagging system - Abstraction enables future persistence without changing consumers - Audit trail fields included on Move struct

---

Core Smart Contract Adapter

Task Group 4: ProductAction and Call 1 Interface

Dependencies: Task Groups 1, 2 **Complexity:** L

- [ ] 4.0 Complete ProductAction types and Call 1 interface
  - [ ] 4.1 Write 4-6 focused tests for ProductAction
    - Test ActionType enum serialization

    - Test ProductAction construction with all fields
    - Test PositionSelector enum variants
    - Test ProductActionRef lineage tracking
- [ ] 4.2 Define ActionType enum
    - CouponPayment, Settlement, BarrierBreach
    - EarlyExercise, DividendAdjustment, StockSplitAdjustment
    - FixingObserved, HedgeUnwind, HedgeRebalance
- [ ] 4.3 Define ProductAction struct
    - action_type: ActionType
    - amount_per_unit: Option
    - settlement_date: Option
    - details: ActionDetails (enum for type-specific data)
    - source_action_ref: Option (for event chaining)
    - target_selector: Option (for downstream targeting)
- [ ] 4.4 Define PositionSelector enum
    - SameAsSource (default)
    - LinkedHedges { source_product_id: ProductId }
    - Specific { position_ids: Vec }
    - ByQuery { filter: PortfolioFilter }
- [ ] 4.5 Define ProductActionCalculator trait (Call 1 interface)
    - fn get_product_action(&self, trigger_info: &TriggerInfo, product: &Product, quant_lib: &dyn QuantLib) -> Result
- [ ] 4.6 Implement serde + prost serialization for ProductAction types
- [ ] 4.7 Ensure ProductAction tests pass
    - Run ONLY the 4-6 tests written in 4.1

Acceptance Criteria: - All ActionTypes cover product lifecycle events - ProductAction supports event chaining via source_action_ref - PositionSelector enables flexible downstream targeting - Call 1 interface is clean and stateless

---

Task Group 5: MoveRules Engine

Dependencies: Task Groups 3, 4 **Complexity:** L

- [ ] 5.0 Complete MoveRules engine
    - [ ] 5.1 Write 4-6 focused tests for MoveRules
        - Test RuleCondition matching (Equals, In, Exists, Not)
        - Test DirectBooking pattern application
        - Test WashBookRouting pattern (3-leg entry)
        - Test Split pattern with percentage allocation
    - [ ] 5.2 Define MoveRuleContext struct
        - attributes: HashMap<String, String>
        - Support dimensions: legal_entity, product_type, jurisdiction, accounting_standard, business_line, client_type, book_type
    - [ ] 5.3 Define RuleCondition enum
        - Equals { attribute: String, value: String }
        - In { attribute: String, values: Vec }
        - Exists { attribute: String }
        - Not(Box)
    - [ ] 5.4 Define BookingPattern enum (6 patterns)
        - DirectBooking { debit_account, credit_account }
        - WashBookRouting { source, wash_book, destination }
        - Split { entries: Vec }

    - TaxWithholding { gross, tax, net, tax_rate_lookup }
    - Composite { patterns: Vec }
    - Custom { rule_id, parameters }
  - [ ] 5.5 Define MoveRule struct
    - conditions: Vec
    - booking_pattern: BookingPattern
  - [ ] 5.6 Implement rule matching logic
    - Match context against RuleConditions
    - All conditions must match (AND logic)
  - [ ] 5.7 Implement booking pattern application
    - Generate Move entries based on pattern type
    - Scale amounts by position quantity
    - Populate audit trail fields
  - [ ] 5.8 Ensure MoveRules tests pass
    - Run ONLY the 4-6 tests written in 5.1

Acceptance Criteria: - Multi-dimensional context matching works correctly - DirectBooking, WashBookRouting, Split patterns generate correct moves - Audit trail fields populated on generated moves - Rule matching is deterministic and testable

---

Task Group 6: Call 2 Interface and ApplyResult

Dependencies: Task Groups 3, 4, 5 **Complexity:** L

- [ ] 6.0 Complete Call 2 interface with event chaining
  - [ ] 6.1 Write 4-6 focused tests for Call 2
    - Test ApplyResult construction with moves and downstream_actions
    - Test MoveApplicator with DirectBooking (single position)
    - Test MoveApplicator with WashBookRouting pattern
    - Test downstream action generation (barrier breach -> settlement)
  - [ ] 6.2 Define PositionWithContext struct
    - position: Position
    - context: MoveRuleContext
    - applicable_rules: Vec (pre-matched by Event Framework)
  - [ ] 6.3 Define ApplyResult struct
    - moves: Vec
    - downstream_actions: Vec
  - [ ] 6.4 Define MoveApplicator trait (Call 2 interface)
    - fn apply_to_positions(&self, product_action: &ProductAction, positions: &[PositionWithContext], move_rules: &[MoveRule]) -> Result
  - [ ] 6.5 Implement MoveApplicator
    - Apply ProductAction to each position
    - Scale amount_per_unit by position quantity
    - Apply matched MoveRules to generate moves
    - Identify downstream ProductActions based on action_type
    - Set source_action_ref for lineage
  - [ ] 6.6 Implement downstream action logic for thin slice
    - BarrierBreach -> Settlement downstream action
    - Populate target_selector appropriately
  - [ ] 6.7 Ensure Call 2 tests pass

- Run ONLY the 4-6 tests written in 6.1

Acceptance Criteria: - ApplyResult properly bundles moves and downstream actions - Event chaining implemented for barrier breach -> settlement - Lineage tracking via source_action_ref works correctly - Position quantities properly scale the ProductAction amounts

---

Thin Slice Implementation

Task Group 7: In-Memory Ledger Implementation

Dependencies: Task Group 3 **Complexity:** M

- ☐ 7.0 Complete in-memory ledger implementation
  - ☐ 7.1 Write 4-6 focused tests for in-memory ledger
    - Test position creation and retrieval
    - Test move recording and balance calculation
    - Test portfolio filtering with tag criteria
    - Test wallet-scoped position queries
  - ☐ 7.2 Implement InMemoryLedger struct
    - positions: HashMap<PositionId, Position>
    - moves: Vec
    - wallets: HashMap<WalletId, Wallet>
  - ☐ 7.3 Implement LedgerReader for InMemoryLedger
    - get_position: O(1) lookup
    - get_positions_by_wallet: filter by wallet_id
    - get_balance: sum moves for position
    - positions_by_portfolio: filter by tag criteria
    - moves_by_portfolio: filter moves by position tags
  - ☐ 7.4 Implement LedgerWriter for InMemoryLedger
    - create_position: generate ID, store position
    - record_moves: append moves, update balances
  - ☐ 7.5 Ensure in-memory ledger tests pass
    - Run ONLY the 4-6 tests written in 7.1

Acceptance Criteria: - InMemoryLedger implements full Ledger trait - Portfolio queries work correctly with tag filtering - Balance calculations aggregate moves correctly - Performance suitable for research/simulation use

---

Task Group 8: Continuous Barrier Products (Thin Slice)

Dependencies: Task Groups 4, 5, 6, 7 **Complexity:** XL

- ☐ 8.0 Complete thin slice with continuous barrier products
  - ☐ 8.1 Write 6-8 focused tests for thin slice
    - Test Knock-Out Warrant barrier breach flow (Call 1 -> ProductAction)
    - Test Knock-Out Warrant move generation (Call 2 -> ApplyResult)
    - Test Mini Certificate barrier breach with event chaining
    - Test fixing trigger (initial fixing sets strike)
    - Test expiry trigger with settlement
    - Test DirectBooking and WashBookRouting patterns on knockout
  - ☐ 8.2 Implement KnockOutWarrant product handler

- Process ContinuousBarrier trigger -> BarrierBreach action
- Process Fixing trigger -> FixingObserved action
- Process Expiry trigger -> Settlement action
- Generate Settlement downstream action on barrier breach

☐ 8.3 Implement MiniCertificate product handler
- Similar to KnockOutWarrant with Mini-specific logic
- Continuous barrier monitoring
- Event chaining on knockout

☐ 8.4 Configure MoveRules for thin slice (at least 3 patterns)
- DirectBooking: Standard knockout/settlement
- WashBookRouting: Intercompany settlement
- Split: Multi-desk P&L allocation

☐ 8.5 Implement full two-call flow
- Call 1: TriggerInfo + Product -> ProductAction
- Call 2: ProductAction + Positions + Rules -> ApplyResult
- Event chaining: Process downstream_actions recursively (simulated)

☐ 8.6 Implement JSON and Protobuf serialization for output
- Moves serializable to JSON via serde
- Moves serializable to Protobuf via prost
- ApplyResult fully serializable

☐ 8.7 Ensure thin slice tests pass
- Run ONLY the 6-8 tests written in 8.1

Acceptance Criteria: - Knock-Out Warrant and Mini Certificate fully functional - Two-call pattern working end-to-end - Event chaining (barrier -> settlement) working - At least 3 booking patterns demonstrated - JSON and Protobuf serialization working

---

Stubs and Integration

Task Group 9: Stubs (Event Framework, Quant Lib)

Dependencies: Task Groups 2, 4 **Complexity:** L

☐ 9.0 Complete stubs for external dependencies

☐ 9.1 Write 4-6 focused tests for stubs
- Test EventFrameworkStub trigger registration
- Test EventFrameworkStub callback dispatch simulation
- Test QuantLibStub product action calculation
- Test two-call orchestration flow

☐ 9.2 Define QuantLib trait
- fn get_action(&self, product: &Product, trigger_data: &TriggerInfo) -> Result

☐ 9.3 Implement QuantLibStub
- Return mock ProductActions for thin slice products
- Support all trigger types needed for testing

☐ 9.4 Define EventFramework trait
- fn register_trigger(&mut self, registration: TriggerRegistration) -> Result<()>
- fn simulate_trigger(&self, trigger_info: TriggerInfo) -> Result<()>

☐ 9.5 Implement EventFrameworkStub
- Store trigger registrations
- Simulate trigger dispatch with mock positions

    - Build mock MoveRuleContext for positions
    - Pre-match MoveRules based on context
    - Orchestrate two-call pattern
    - Handle downstream_actions recursively (with max depth)
  - ☐ 9.6 Implement recursion safety in stub
    - Max depth limit (10 levels)
    - Cycle detection (action_type + product_id)
  - ☐ 9.7 Ensure stub tests pass
    - Run ONLY the 4-6 tests written in 9.1

Acceptance Criteria: - QuantLibStub returns realistic ProductActions - EventFrameworkStub orchestrates full two-call flow - All 8 trigger types can be simulated - Recursion safety implemented for event chaining

---

Task Group 10: Portfolio Tagging System

Dependencies: Task Groups 3, 7 **Complexity:** M

- ☐ 10.0 Complete portfolio tagging system (GAP from Python POC)
  - ☐ 10.1 Write 4-6 focused tests for portfolio system
    - Test TagCriterion matching (Equals, In, Exists)
    - Test PortfolioFilter with multiple criteria (AND logic)
    - Test position appearing in multiple portfolio queries
    - Test move tag inheritance from position
  - ☐ 10.2 Define TagKey and TagValue types
    - TagKey: String (e.g., “strategy”, “desk”, “trader”)
    - TagValue: String (e.g., “yield_enhancement”, “desk_a”)
  - ☐ 10.3 Define TagCriterion enum
    - Equals(TagKey, TagValue)
    - In(TagKey, Vec)
    - Exists(TagKey)
  - ☐ 10.4 Define PortfolioFilter struct
    - name: String (for named portfolio views)
    - criteria: Vec (AND logic)
  - ☐ 10.5 Implement tag inheritance
    - Wallet default tags copied to new positions
    - Position tags override wallet defaults
    - Move inherits position tags on creation
  - ☐ 10.6 Implement portfolio query methods on InMemoryLedger
    - positions_by_portfolio: filter positions by tag criteria
    - moves_by_portfolio: filter moves by inherited tags
  - ☐ 10.7 Ensure portfolio system tests pass
    - Run ONLY the 4-6 tests written in 10.1

Acceptance Criteria: - Flexible tagging system for multi-dimensional classification - Non-exclusive portfolio membership (position in multiple portfolios) - Portfolio queries compute membership at runtime - Move tag inheritance enables P&L attribution

---

Test Review & Integration

Task Group 11: Test Review & Integration Testing

Dependencies: Task Groups 1-10 **Complexity:** M

- [ ] 11.0 Review tests and add critical integration coverage
  - [ ] 11.1 Review existing tests from Task Groups 1-10
    - Verify 4-6 tests per group (approximately 48-60 tests total)
    - Identify critical integration gaps
  - [ ] 11.2 Analyze integration test gaps for THIS feature only
    - Focus on end-to-end two-call pattern flows
    - Focus on event chaining scenarios
    - Focus on multi-pattern booking scenarios
    - Do NOT assess entire application test coverage
  - [ ] 11.3 Write up to 8 additional integration tests
    - End-to-end: Continuous barrier breach -> moves + downstream settlement
    - End-to-end: Fixing -> settlement flow
    - Integration: Two-call pattern with 3 different booking patterns
    - Integration: Event chaining with lineage tracking
    - Integration: Portfolio-scoped move queries
    - Serialization: Full ApplyResult to JSON roundtrip
    - Serialization: Full ApplyResult to Protobuf roundtrip
    - Stub orchestration: Complete trigger -> moves flow
  - [ ] 11.4 Run all feature-specific tests
    - Run tests from Groups 1-10 plus new integration tests
    - Expected total: approximately 56-68 tests
    - Verify all critical workflows pass

Acceptance Criteria: - All unit tests from Task Groups 1-10 pass - All integration tests pass - Two-call pattern validated end-to-end - Event chaining validated with lineage - JSON and Protobuf serialization validated

---

Execution Order

Recommended implementation sequence based on dependencies:

```
Phase A: Foundation (Groups 1-3)
  [Group 1: Domain Model] --> [Group 2: Trigger Types]
                          --> [Group 3: Ledger Abstraction] (MUST
come first!)

Phase B: Core Adapter (Groups 4-6)
  [Group 4: ProductAction/Call 1] --> [Group 5: MoveRules] -->
[Group 6: Call 2/ApplyResult]

Phase C: Thin Slice (Groups 7-8)
  [Group 7: InMemory Ledger] --> [Group 8: Continuous Barrier
Products]

Phase D: Integration (Groups 9-11)
  [Group 9: Stubs] --> [Group 10: Portfolio Tagging] --> [Group 11:
Integration Tests]
```

Critical Path: 1. Group 1 (Domain Model) - enables all subsequent work 2. Group 3 (Ledger Abstraction) - MUST precede Group 7 (InMemory Ledger) 3. Group 8 (Thin Slice) - validates entire architecture

---

Notes

Key Architectural Decisions

- **Ledger Abstraction First**: Group 3 must be completed before Group 7 to ensure consistent interface
- **Two-Call Pattern**: Call 1 (product-level) is stateless, Call 2 (position-level) applies booking rules
- **Event Chaining**: ApplyResult contains both moves AND downstream ProductActions
- **Stateless Design**: This library has no internal trigger registry or position storage

Thin Slice Rationale

- Knock-Out Warrant and Mini Certificate chosen because continuous barrier logic already exists in Event Framework
- Validates most complex trigger type first (continuous price monitoring)
- Time-based triggers (coupon, expiry) are simpler and can be added after

Out of Scope (per requirements)

- Event Framework implementation (stub only)
- Quant Lib implementation (stub only)
- Market data service (prices passed in callbacks)
- Persistence/database layer
- UI/API layer
- Recursion handling in production (Event Framework responsibility)

Testing Strategy

- 4-6 focused tests per task group during development
- Maximum 8 additional integration tests for gap analysis
- Test behavior, not implementation
- Mock external dependencies (stubs)